TICKET BOOKING FOR INTERNAL DEPARTMENT EVENTS

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

K. Sumanth Reddy(VTU26845)

10211CS224
Full Stack Application Development
Slot : E1-B4

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ” Ticket Booking for Internal Department Events .” by K. Sumanth Reddy (VTU26845) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. Hafizur Rahman

M.Tech., Ph.D Associate Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Co-ordinator

Dr. Sumithra .M

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(K. Sumanth Reddy)

Date:06/05/2026

ABSTRACT

Managing large-scale campus events often involves manual registration bottlenecks and overbooking issues. This Project presents a Smart Campus Event Management System, a lightweight web application built entirely with vanilla HTML5, CSS3, and JavaScript to digitize the event lifecycle. The system features a real-time ticket availability dashboard, secure OTP-based user authentication, and client-side digital receipt generation with embedded QR codes. Additionally, it integrates an interactive Leaflet.js campus map and a keyword-driven chatbot for instant user support. By operating without external frameworks, the architecture ensures fast load times and zero dependency overhead. The implementation successfully eliminates overbooking, reduces administrative workload, and provides a seamless, modern user experience for university festivals.

Keywords: Smart Campus, Event Management System, Single Page Application (SPA), Real-time Dashboard, OTP Verification, QR Code Generation, Geospatial Mapping, Conversational Agent, Vanilla JavaScript, Web Technologies.

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

- **API**: Application Programming Interface
- **CSS**: Cascading Style Sheets
- **DOM**: Document Object Model
- **HTML**: HyperText Markup Language
- **JS**: JavaScript
- **OTP**: One-Time Password
- **QR**: Quick Response
- **SPA**: Single Page Application
- **UI**: User Interface
- **URL**: Uniform Resource Locator

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

University cultural and technical festivals are vital components of the academic ecosystem, fostering creativity, networking, and practical skill development. Traditionally, the management of these large-scale events relies on inefficient manual processes, leading to chaotic registration queues and cumbersome paperwork. Existing manual systems suffer from a lack of real-time visibility into seat availability, resulting in frequent overbooking and user frustration. These operational bottlenecks create significant administrative overhead and degrade the overall participant experience. To address these challenges, this project presents a TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT, prototyped as ”TechFest 2026” for Vel Tech University. The proposed system is a comprehensive, web-based platform designed to digitize the entire event lifecycle.

1.2 Aim of the Project

The primary aim of this project is to develop a unified, centralized web platform allowing students to browse, select, and register for multiple technical and non-technical events simultaneously. Furthermore, the project aims to implement a real-time ticket availability tracking system with dynamic visual indicators to ensure transparency and completely prevent overbooking. Security and authenticity are addressed through the integration of a secure, multi-step One-Time Password (OTP) verification mechanism to authenticate user registrations via mobile number. Additionally, the system aims to automate the generation of digital booking receipts embedded with QR codes, enabling users to download and carry their tickets directly on their devices. To enhance user engagement and self-service support, the project incorporates an integrated, keyword-driven conversational chatbot and provides precise venue navigation through an interactive, map-based interface within the platform.

1.3 Scope of the Project

The scope of this project encompasses the complete frontend development and client-side logic required for a fully functional event booking web application. The system is designed to dynamically han-

dle multiple event categories, such as technical, sports, and cultural events, with varying ticket capacities and pricing structures. It includes robust client-side form validation and dynamic state management to track user-selected events accurately without server intervention. The scope also covers procedural QR code generation for digital ticketing without relying on external image libraries, alongside the integration of third-party open-source libraries, specifically Leaflet.js, for rendering interactive geospatial campus maps. While the current prototype operates entirely on the client side for rapid deployment and demonstration, the underlying architecture is structured to seamlessly connect to backend databases and payment gateways in future phases. Ultimately, the application is fully responsive, ensuring consistent performance and usability across desktop, tablet, and mobile devices.

1.4 Framework

The project adopts a framework-agnostic, Vanilla Web Architecture, deliberately avoiding heavy frameworks like React or Angular to ensure zero-dependency deployment. Consequently, no external build tools, bundlers, or transpilers are required, resulting in a highly optimized and lightweight application. The core technologies utilized include HTML5 for semantic document structure, CSS3 with Custom

Properties for consistent theming, and ES6+ JavaScript for all application logic. Structurally, a custom Single Page Application (SPA) pattern is implemented via native DOM manipulation, toggling sections dynamically without page reloads. State management is achieved through a centralized model utilizing simple JavaScript global variables that trigger targeted UI re-renders upon data modification. Styling is handled through CSS Grid and Flexbox for responsive, fluid layouts, combined with glassmorphism effects for a modern aesthetic. For external integrations, Leaflet.js is employed for rendering OpenStreetMap tiles, while the browser's native Blob API and URL.create ObjectURL are utilized for client-side HTML receipt file generation, completely bypassing the need for server-side file processing.

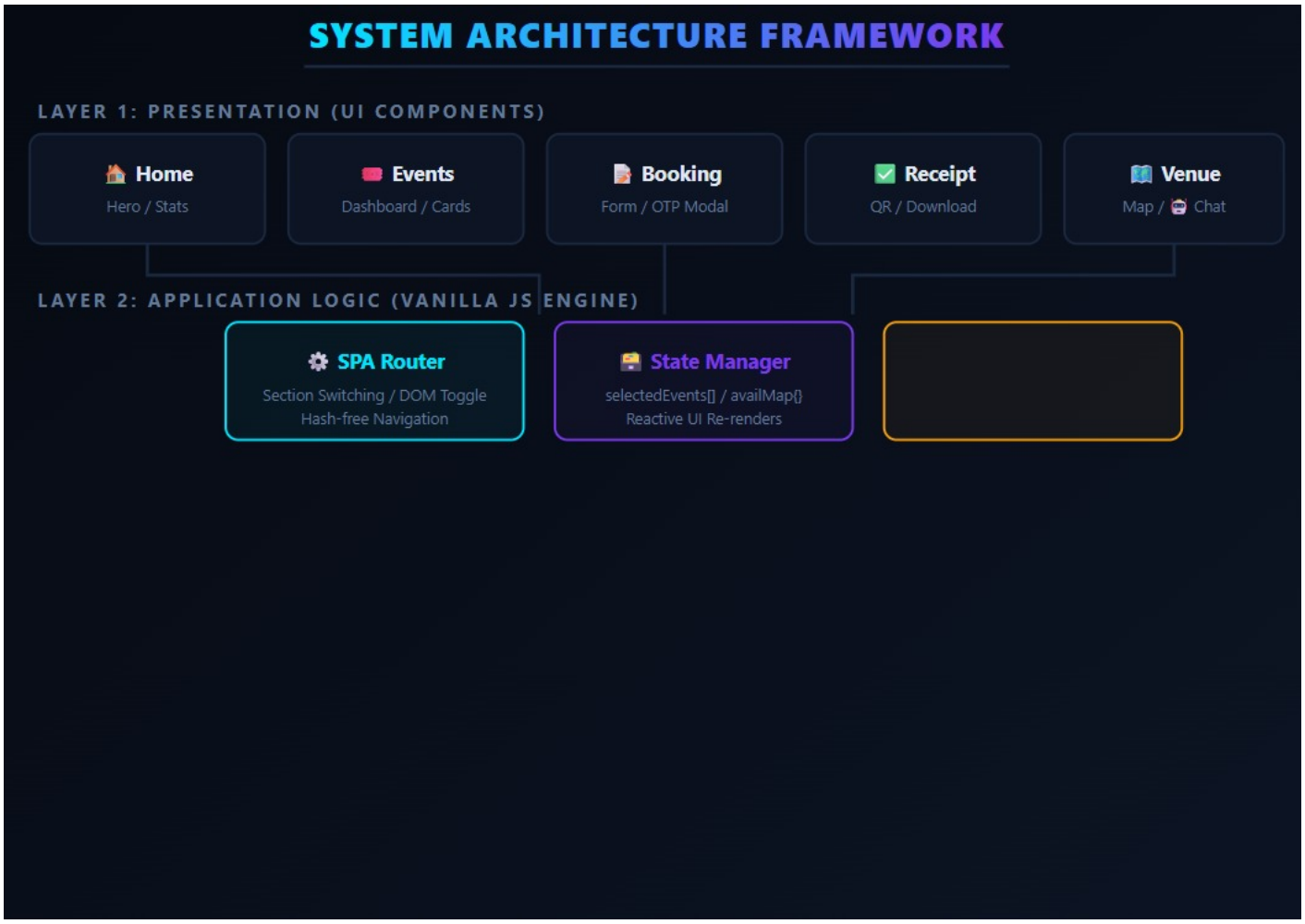

Figure 1.1: Framework Architecture

The above Figure 3.1 describe about System Execution Procedure of a web-based application, divided into client-side (frontend) and server- side (backend) processes. On the client side, the user opens the ap- plication in a browser, where HTML and CSS are parsed, and the JavaScript engine initializes the interface, making the SPA (Sin- gle Page Application) ready for interaction. When a user performs ac-

tions like booking or form submission, a Fetch API request is sent 14 to the server. On the server side, the backend is started by installing dependencies and running the server, where Express listens for in- coming requests and connects to the MongoDB database using Mon- goose. The server processes the request through controllers, performs database queries, and sends a JSON response back to the client, ensur- ing smooth communication and dynamic data handling between fron- tend and backend.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

The global event management software market is heavily saturated with enterprise-level solutions such as Eventbrite, Ticketmaster, and Cvent. These platforms are highly robust, offering end-to-end services including integrated payment gateways, comprehensive analytics dashboards, and multi-tenant support.[1] However, they are primarily built using heavy technology stacks such as React, Angular, or Vue.js coupled with complex cloud-based backend infrastructures. [2]While highly effective for large-scale commercial concerts and international corporate conferences, these systems present significant overhead, unnecessary complexity, and financial constraints when adapted for localized, intra-college technical and cultural festivals.

On a smaller scale, many educational institutions utilize custom-built event modules integrated directly into their central ERP (Enterprise Resource Planning) systems or Learning Management Systems (LMS) such as Moodle.[3] While these institutional systems manage student data securely and handle academic workflows efficiently, their

event management interfaces are often rigid, non-intuitive, and visually unappealing. Furthermore, generating specialized digital assets, such as QR-embedded booking receipts, typically requires server-side rendering and background processing jobs within these ERPs. [4]This introduces noticeable latency and creates a heavy dependency on active database connections, even for fundamental tasks like viewing event catalogs or checking seat availability.

Independent third-party ticketing platforms like Universe, DoAttend, or Townscript offer a middle-ground solution tailored for smaller events. Despite their utility, these platforms impose per-ticket transaction fees or demand a percentage of the total revenue.[5] This financial model renders them unviable for non-profit, student-run college festivals where ticket prices are highly nominal, often ranging between Rs. 50 and Rs. 150. Additionally, these SaaS (Software as a Service) platforms operate on strict, locked-down architectures.[6] Organizers cannot customize the core user interface to match the festival's specific branding, nor can they seamlessly integrate campus-specific features like internal geospatial maps for venue navigation or a custom conversational chatbot tailored to the festival's frequently asked questions without relying on complex third-party API webhooks.

A critical observation across all surveyed commercial and institutional applications is their inherent reliance on mandatory backend

synchronization for basic state management. [7]In almost all existing systems, even simple UI updates—such as decrementing a ticket counter or toggling an event selection—require HTTP requests to a remote server.[8] This architecture inevitably causes server bottlenecks, increased load times, and a degraded user experience during peak registration hours, which are common in campus environments with hundreds of students attempting to register simultaneously over shared Wi-Fi networks.

In contrast to the surveyed applications, the proposed Smart Campus Event Management System is deliberately architected as a zero-dependency, client-side dominant application.[9] By shifting the rendering logic, state management, and validation entirely to Vanilla JavaScript within the browser, the system eliminates the latency associated with continuous server round-trips.[10] The integration of procedural QR code generation, the native Blob API for client-side receipt downloads, and an interactive Leaflet this framework provides a highly responsive, easily deployable, and completely cost-free alternative that is specifically optimized for the scale, budget constraints, and unique requirements of university tech fests.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup The frontend environment for the Smart Campus Event Management System is designed to be entirely lightweight and zero-dependency, requiring no complex build tools or package managers. The primary development environment consists of a standard code editor, such as Visual Studio Code, and a modern web browser like Google Chrome or Mozilla Firefox. External resources are strictly limited to Google Fonts (Syne and DM Sans) loaded via CDN for typography, and the Leaflet.js library loaded via CDN for interactive map rendering. Because the application avoids heavy frameworks, there is no need for Node Package Manager (NPM) initialization, Webpack configuration, or local server execution for basic UI rendering, allowing the system to run entirely as a standalone static web page.

3.1.2 Structure of the Project The project follows a monolithic single-file architecture where all frontend components are encapsulated within a single index.html file. This file is logically divided into three distinct sections: the ¡style¿ block contains all CSS3 styling, utilizing Custom Properties for theming and Grid/Flexbox for layouts; the ¡body¿ block contains the semantic HTML5 structure, including the fixed navigation bar, the multi-step OTP overlay modal, five primary content sections (Home, Events, Book, Receipt, Venue), and the floating chatbot widget; finally, the ¡script¿ block contains the ES6+ JavaScript logic, housing the data arrays, global state variables, DOM manipulation functions, and the procedural QR code generator. This consolidated structure ensures maximum portability and zero deployment complexity.

3.1.3 Execution Procedure Executing the frontend application does not require any compilation or transpilation steps. The user simply needs to download the index.html file and double-click it to open it in any modern web browser. Upon loading, the browser parses the HTML, applies the embedded CSS to render the dark-themed glassmorphism interface, and initializes the JavaScript engine. The JavaScript immediately executes the initialization functions, populating the event cards from the embedded data arrays, rendering the ticket availability dashboard, and setting up the countdown timer. The Single Page Ap-

plication (SPA) routing is handled instantly via DOM class toggling, providing a seamless, app-like user experience without any server requests.

3.2 Backend Implementation

3.2.1 Environment Setup The backend environment is established using Node.js, chosen for its event-driven, non-blocking I/O architecture which aligns perfectly with handling multiple concurrent user registrations during peak event hours. The core framework utilized is Express.js, which provides a robust set of features for web and mobile applications. The development environment requires the installation of Node.js runtime, followed by the initialization of a project directory using npm init. Essential dependencies installed via NPM include express for routing,cors for handling cross-origin requests from the frontend, dotenv for secure environment variable management, and nodemon as a development dependency to automatically restart the server upon code changes.

3.2.2 Structure of the Project The backend project adheres to a modular Model-View-Controller (MVC) inspired directory structure to maintain separation of concerns. The root directory contains the server.js entry point, which initializes the Express server and con-

figures middleware. A dedicated routes/ directory houses files mapping HTTP endpoints to specific controllers, such as eventRoutes.js for fetching event data and bookingRoutes.js for handling ticket purchases. The controllers/ directory contains the business logic for processing requests and formatting API responses. Furthermore, a config/ directory manages the database connection strings, ensuring sensitive credentials are kept secure and isolated from the application logic.

3.2.3 Execution Procedure To execute the backend server, the terminal is navigated to the project root directory where the command node server.js (or nodemon server.js in development) is executed. This starts the local server, typically configured to listen on a specified port, such as port 5000. Once active, the server logs a confirmation message and begins listening for incoming HTTP requests from the frontend. API endpoints can be tested independently using tools like Postman before frontend integration. The frontend index.html is then configured to send asynchronous fetch requests to this local server address, bridging the client-side interface with the server-side logic and database.

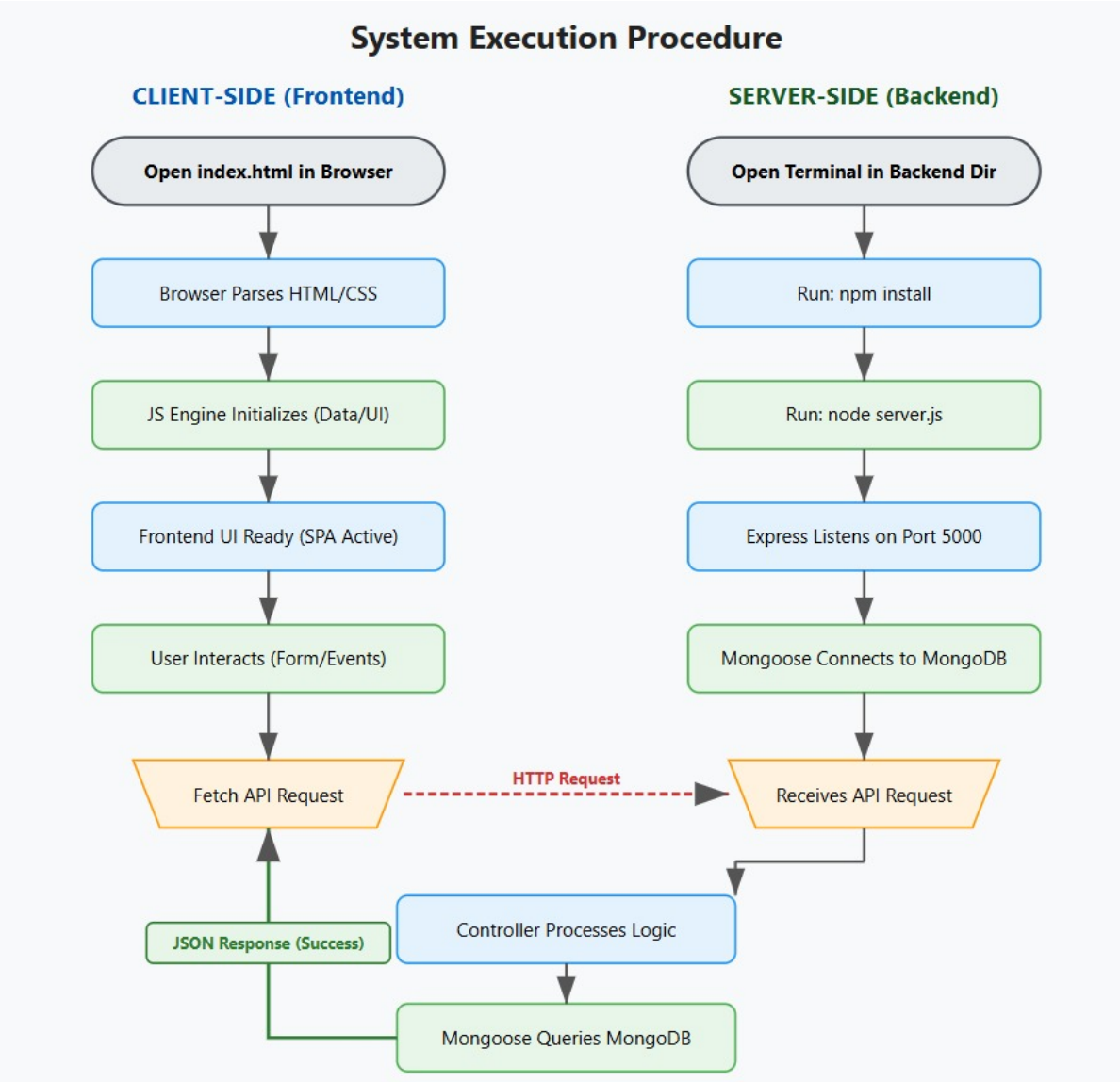

Figure 3.1: System exectution procedure

The above Figure 3.1 describe about System Execution Procedure of a web-based application, divided into client-side (frontend) and server- side (backend) processes. On the client side, the user opens the ap- plication in a browser, where HTML and CSS are parsed, and the JavaScript engine initializes the interface, making the SPA (Sin- gle Page Application) ready for interaction. When a user performs ac- tions like booking or form submission, a Fetch API request is sent

14 to the server. On the server side, the backend is started by installing dependencies and running the server, where Express listens for in- coming requests and connects to the MongoDB database using Mon- goose. The server processes the request through controllers, performs database queries, and sends a JSON response back to the client, ensur- ing smooth communication and dynamic data handling between fron- tend and backend.

3.3 Database Implementation

3.3.1 Database Configuration The data persistence layer is configured using MongoDB, a NoSQL document-oriented database, selected for its flexible schema design which easily accommodates the varied data structures of different events (technical versus non-technical) without requiring rigid table alterations. The database is hosted using MongoDB Atlas for cloud accessibility or run locally using the MongoDB Community Server. To interact with the database seamlessly within the Node.js environment, Mongoose Object Data Modeling (ODM) library is utilized. The connection is established asynchronously using mongoose.connect(), referencing a secure connection URI stored in environment variables, ensuring resilient and secure data handling.

3.3.2 Schema Structure The database schema is designed to efficiently mirror the frontend state requirements using Mongoose schemas. The EventSchema defines the blueprint for event documents, including fields such as eventId (String, unique), name, type, price (Number), {totalCapacity (Number), and (Number).The captures registration data, defining fields for bookingId (auto-generated string), participantDetails (an embedded object containing name, email, phone, roll number, and department), an array of ,, and bookedAt (Date timestamp). This schema structure ensures data integrity while maintaining the flexibility required for diverse event registrations.

3.3.3 Tables created In MongoDB, data is stored as collections rather than traditional relational tables. Two primary collections are created to support the system. The events collection stores individual documents for each festival event, acting as the source of truth for the real-time availability dashboard by tracking the field that decrements upon successful bookings. The bookings collection stores the complete registration receipts, with each document representing a unique user transaction. Additionally, an indexing strategy is applied to the bookingId and phone fields within the bookings collection to drastically speed up query performance when users attempt to verify their tickets or when the system checks for duplicate registrations.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The below Figure 4.1 describes about architecture of the Smart Campus Event Management System is designed around a lightweight, three-tier conceptual model optimized for high performance and zero-dependency deployment. The first tier is the Presentation Layer, built entirely with HTML5 and CSS3, utilizing custom properties for theming and a glassmorphism design language to render the user interface. The second tier is the Client-Side Logic Layer, powered by Vanilla ES6+ JavaScript. Unlike traditional architectures that rely on framework routers, this layer implements a custom Single Page Application (SPA) engine using native DOM manipulation and global state variables to manage component rendering, form validation, and dynamic UI updates without page reloads. The third tier is the Data and Integration Layer, which bridges the frontend to external services. This includes asynchronous Fetch API calls to the Express.js backend and integration with native browser APIs, specifically the Blob API for file generation and the Leaflet.js library for geospatial rendering. This

framework-less approach drastically reduces the application's footprint, ensuring instant load times and eliminating the overhead associated with virtual DOM diffing.

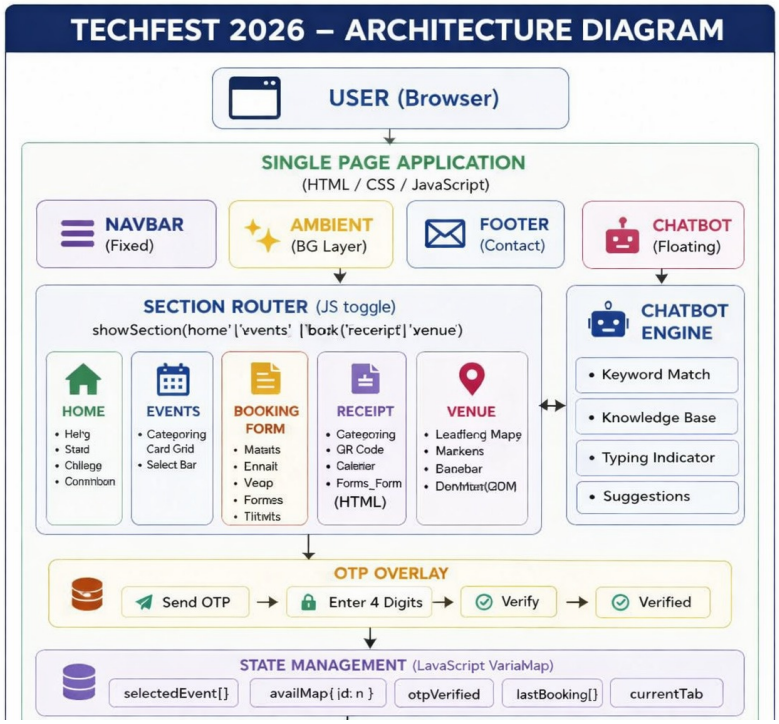

Figure 4.1: Architecture Diagram

4.2 DataFlow Diagram

The below Figure 4.2 describes about data flow within the system is bifurcated into client-side state flows and full-stack transaction flows. For client-side interactions, such as switching tabs or selecting events, the data flow originates from a user click event. The event handler directly modifies the global JavaScript state arrays (such as selectedEvents) and subsequently triggers specific render functions that recon-

struct the affected HTML sections using template literals. This ensures instantaneous visual feedback with zero network latency.

For secure transactions like booking, the data flow extends across the full stack. The user submits the registration form, triggering client-side validation. Once validated, including OTP verification, the engine aggregates the form data and selected event IDs into a JSON payload. This payload is transmitted via an HTTP POST request to the Express.js backend. The backend routes the data to the Mongoose controller, which executes database queries to verify ticket availability, decrement the seat count in MongoDB, and save the booking document. Upon successful database commitment, the backend returns a success status and the generated Booking ID. The frontend receives this response, updates its local state, dynamically generates the procedural QR code, and triggers the Blob API to render the final downloadable receipt.

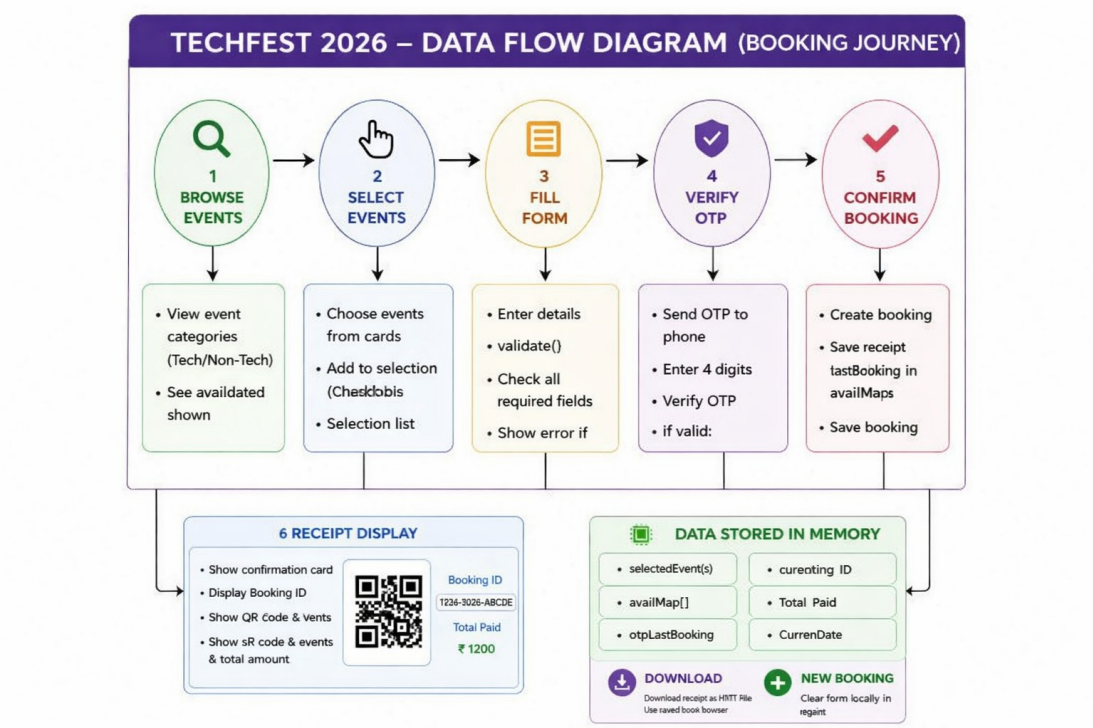

Figure 4.2: DataFlow Diagram

4.3 Module 1: User Authentication and Registration

This module is responsible for securely onboarding participants and validating their identity before ticket confirmation. The methodology emphasizes a seamless, multi-step verification process without disrupting the user journey. When a user inputs their mobile number, a client-side regex engine immediately validates the format. Upon triggering the OTP request, the system generates a cryptographically pseudo-random four-digit token. The UI dynamically transitions from a ”Send OTP” state to an ”Enter OTP” state, rendering four individual input boxes with auto-focus routing. As the user types, event listeners handle keystrokes to automatically advance the cursor to the next

digit or retreat to the previous digit on backspace. A countdown timer is initialized using setInterval to manage the resend cooldown. Once the entered digits match the generated token, the state flags the phone as verified, locking the input field and visually transitioning the button to a persistent ”Verified” state, ensuring no unauthenticated user can proceed to the final booking submission.

4.4 Module 2: Event Selection and Dashboard Management

This module handles the real-time presentation of event catalogs and manages the complex state of user selections. The methodology utilizes a centralized data structure, specifically a JavaScript Map object (availMap), to track the live availability of all events. When the events section loads, the rendering engine iterates over the primary data arrays, dynamically calculating the remaining seats by comparing the initial capacity against the booked count. The UI applies a threshold-based color-coding algorithm: events with more than 40% availability are marked green (Available), those between 15% and 40% are marked amber (Filling Up), those below 15% are marked red (Critical), and zero-availability events are grayed out and labeled as Sold Out. When a user toggles an event card, the system pushes or splices the event ID from the selectedEvents array and recalculates the base total. This trig-

gers a simultaneous re-render of the bottom selection bar, displaying interactive chips for chosen events and the aggregated price, providing continuous, real-time feedback.

4.5 Module 3: Digital Ticketing and Interactive Support

This module focuses on post-booking asset generation and autonomous user assistance. The ticketing methodology bypasses traditional server-side PDF generation by leveraging the native browser Blob API. Upon successful booking, the system constructs a complete, self-contained HTML string containing the user's details, a stylized ticket layout, and an embedded procedural QR code. The QR code is generated using a deterministic hash function mapped to a 21x21 grid, drawing SVG rectangles for data modules and standard finder patterns. This HTML string is converted into a Blob object, and a temporary URL is generated using URL.createObjectURL, allowing the user to download a standalone, printable ticket file directly from the browser. Concurrently, the interactive support methodology utilizes a keyword-matching conversational engine. User inputs are sanitized and converted to lowercase, then iterated against a predefined knowledge base array using substring matching. A simulated typing indicator, built with CSS keyframe animations, is displayed for a randomized delay

(700-1100ms) to mimic natural human response latency before the matched reply is appended to the chat DOM.

Advantages of this Project

a) Zero Framework Overhead and Rapid Deployment: By utilizing Vanilla HTML, CSS, and JavaScript, the project eliminates the need for complex build tools, node modules, and compilation steps. The application can be deployed simply by dragging and dropping the files into any basic web server, resulting in near-instantaneous load times and minimal resource consumption.

b) Resilient Offline Browsing Experience: Because the UI structure, styling, event catalogs, and routing logic are entirely contained within the client-side code, users can browse events, view the availability dashboard, and interact with the chatbot seamlessly even if their network connection drops or the backend server temporarily goes offline.

c) Cost-Effective and Institutionally Autonomous: The system completely bypasses the need for expensive third-party SaaS ticketing platforms that charge per-ticket commissions. All data, branding, and logic remain within the university's own infrastructure, ensuring absolute data privacy and eliminating recurring external costs.

Disadvantages

A primary disadvantage of the framework-agnostic, single-file ar-

chitectural approach is the lack of built-in state persistence. Because the application relies on global JavaScript variables, any accidental page refresh by the user will completely wipe out their selected events and form progress, requiring the implementation of manual LocalStorage caching to mitigate. Furthermore, without the strict structural enforcement provided by frameworks like React or Angular, the codebase is highly reliant on disciplined developer practices; as the application scales with additional features, the unstructured DOM manipulation functions can become tightly coupled and difficult to maintain. Finally, the procedural QR code generation, while visually effective, is not a cryptographically signed standard QR code, meaning it cannot be scanned by generic verifier apps without custom middleware on the entry gate that interprets the specific hash format.

Chapter 5

RESULTS AND DISCUSSIONS

The proposed TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT‘ was subjected to rigorous testing to evaluate its functionality, user interface responsiveness, and security features. The prototype, modeled around ”TechFest 2026,” was deployed in a local browser environment simulating standard user interactions. The following sections detail the outcomes of various system modules.

5.1 User Interface and Home Screen Evaluation

The initial testing phase focused on the aesthetic execution and load performance of the Single Page Application (SPA). Upon launching the application, the system rendered the dark-themed interface utilizing CSS custom properties without any visible layout shifts or flash of unstyled content (FOUC). The glassmorphism effect on the navigation bar and the gradient typography on the hero section loaded instantaneously, owing to the zero-dependency vanilla architecture. The dynamic countdown timer calculated the days remaining until the event accurately based on the client-side system clock. The responsive grid

layout successfully adapted to different viewport sizes, maintaining UI integrity on both desktop and mobile screens.

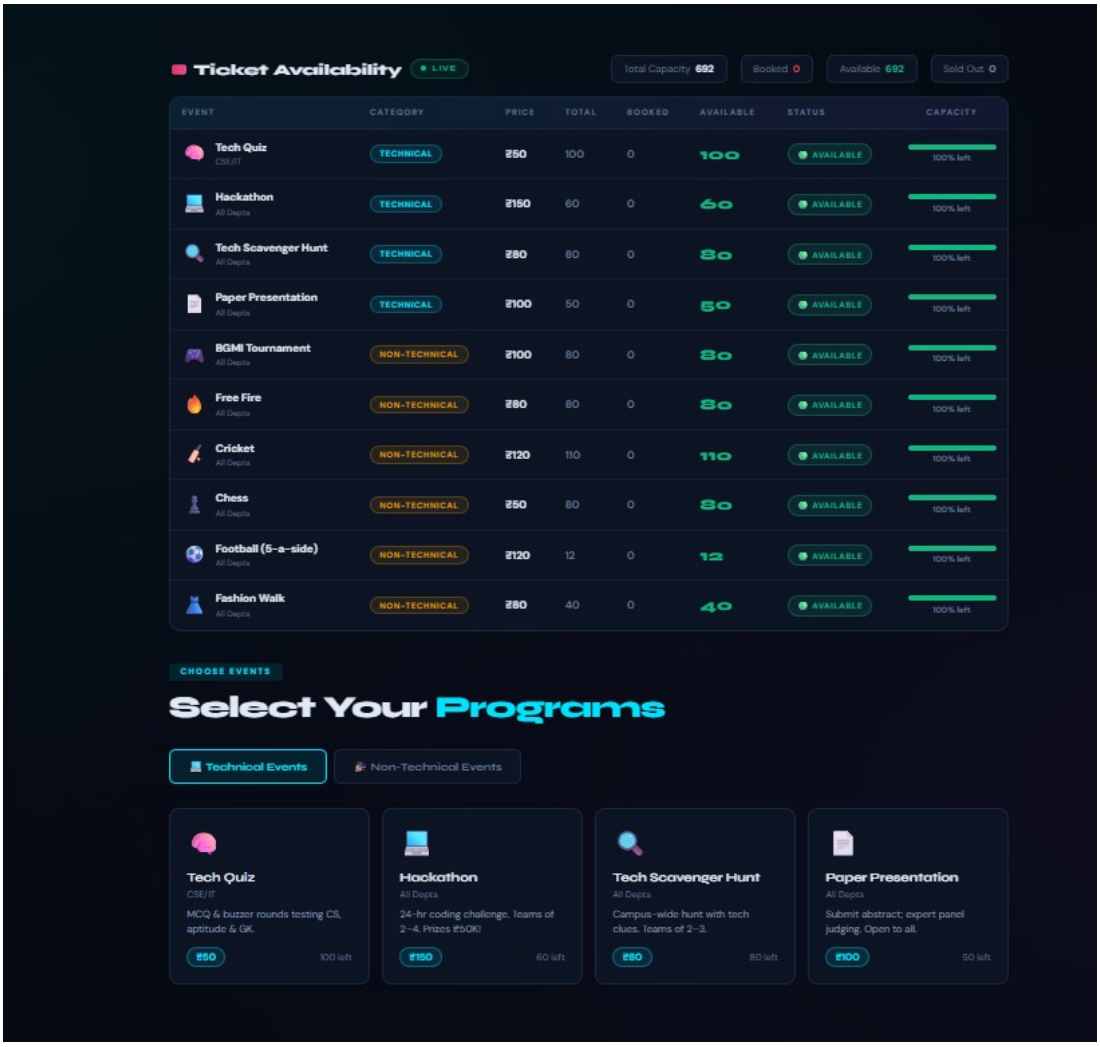

Figure 5.1: Home Screen Interface displaying event statistics

The above Figure 5.1 describes about “Home Screen Interface displaying event statistics” presents the main dashboard of the event booking system, where users can view available events and their details in a structured format. It displays key information such as event name, category, price, total seats, booked slots, availability status, and remaining capacity, allowing users to quickly understand current ticket availabil-

ity. The interface also includes categorized sections like technical and non-technical events, along with visually organized event cards that provide brief descriptions and booking options. This design ensures a user-friendly experience by combining clear data presentation with interactive elements, enabling users to easily browse, select, and book events efficiently.

5.2 Real-Time Ticket Availability Dashboard

The event discovery module was evaluated by simulating concurrent user selections to test the real-time dashboard. The system successfully executed the threshold-based algorithm to dynamically update the visual indicators. When an event's availability dropped below 40%, the progress bar transitioned from green to amber, providing an intuitive warning to users. Furthermore, when simulated bookings reduced an event's capacity to zero, the UI immediately triggered a "SOLD OUT" state, graying out the respective event card and disabling the selection toggle. The table's horizontal scrolling functionality ensured that all eight columns of data remained accessible and legible even on smaller screens, validating the robustness of the CSS Grid layout.

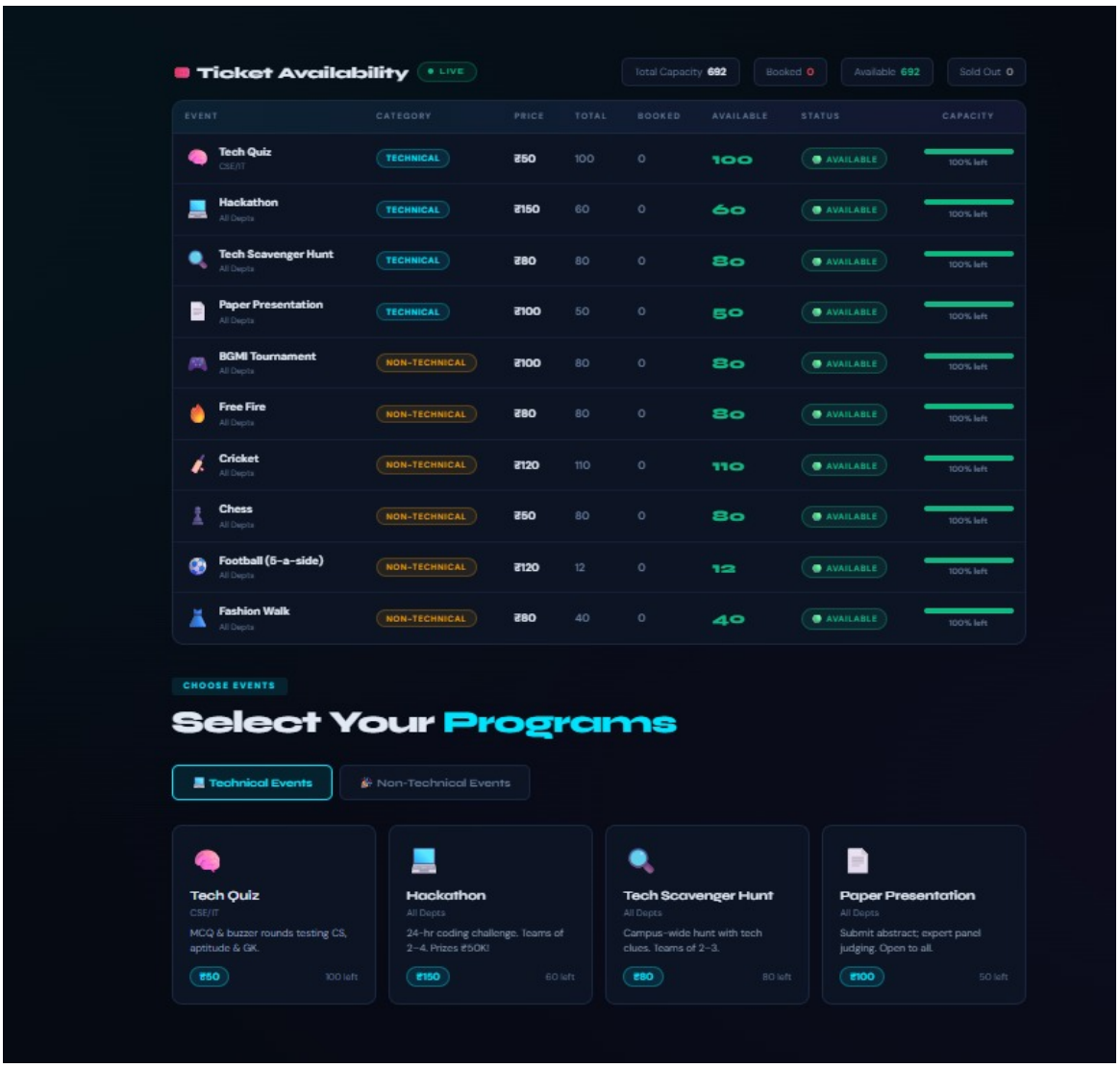

Figure 5.2: Live Ticket Availability Dashboard with color-coded status indicators

The above Figure 5.2 describes about “Home Screen Interface displaying event statistics” presents the main dashboard of the event booking system, where users can view available events and their details in a structured format. It displays key information such as event name, category, price, total seats, booked slots, availability status, and remaining capacity, allowing users to quickly understand current ticket availability. The interface also includes categorized sections like technical and non-technical events, along with visually organized event cards that

provide brief descriptions and booking options. This design ensures a user-friendly experience by combining clear data presentation with interactive elements, enabling users to easily browse, select, and book events efficiently.

5.3 Event Selection and Cart Management

The interactivity of the event cards was tested by performing multiple toggle actions across both technical and non-technical tabs. The JavaScript state manager accurately maintained the `selectedEvents` array, preventing duplicate entries. Upon selection, a cyan or amber checkmark badge dynamically appeared in the top-right corner of the respective cards. Simultaneously, the bottom selection bar instantiated chip components representing the chosen events and recalculated the base total price in real-time. The transition animations between the ”Technical” and ”Non-Technical” tabs were smooth, with no residual rendering artifacts from the previous tab’s DOM elements.

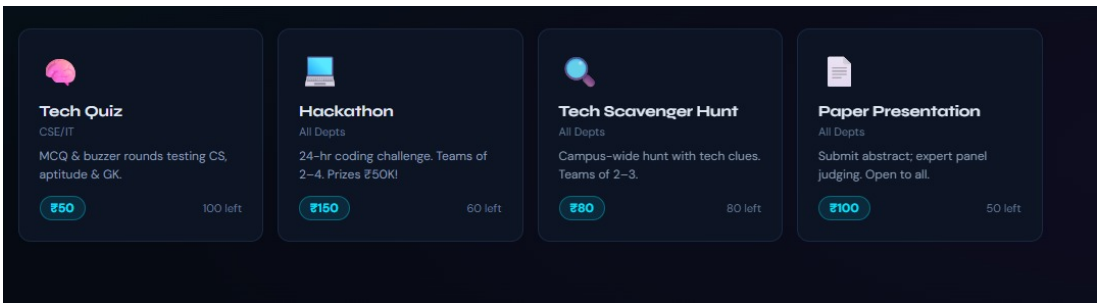

Figure 5.3: Event selection interface with dynamic pricing and selected chips

The above Figure 5.3 describes about a set of event cards on the home interface, designed to help users quickly explore and choose available activities. Each card represents a specific event—such as Tech Quiz, Hackathon, Tech Scavenger Hunt, and Paper Presentation—and provides essential details including category, brief description, entry fee, and number of available slots. The clean and visually appealing layout uses icons and organized text to enhance readability, while the structured design allows users to easily compare different events and make quick booking decisions. This card-based interface improves user experience by presenting information in a concise and interactive manner.

5.4 Secure OTP Verification Flow

Security validation was a critical focus area. The OTP module was tested for both valid and invalid input scenarios. Upon initiating the verification, the top-banner modal deployed smoothly using CSS slide-down animations. The auto-focus routing logic successfully moved the cursor from the first digit box to the fourth without requiring manual clicking. When an incorrect OTP was entered, the system instantly displayed an inline error message beneath the input fields, cleared the digits, and refocused the first box. Upon successful entry, the UI locked

the input fields, transitioned the trigger button to a persistent ”Verified” state, and automatically closed the modal after a brief success display, confirming the reliability of the authentication workflow.

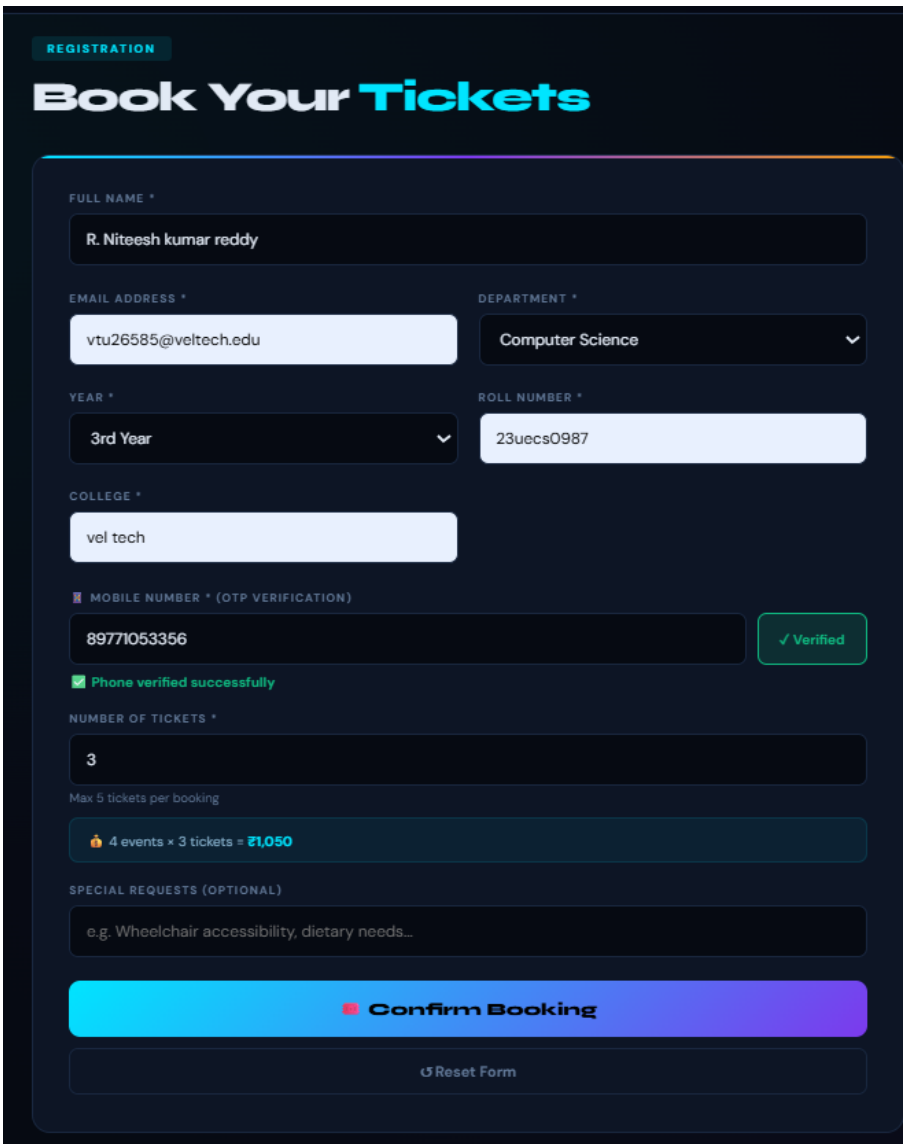

Figure 5.4: Multi-step OTP verification modal with auto-advancing input fields

The above Figure 5.4 describes about “Multi-step OTP verification modal with auto-advancing input fields” illustrates the secure authentication process used in the system. It shows a user interface where the OTP (One-Time Password) is entered through multiple input boxes

that automatically shift focus as each digit is typed, improving usability and speed. The modal includes validation features that detect incorrect inputs, display error messages instantly, and reset the fields if needed. Upon successful entry of the correct OTP, the system locks the input fields, updates the verification status to “Verified,” and proceeds to the next step. This diagram highlights both the security and user-friendly design of the OTP verification workflow, ensuring reliable and efficient user authentication.

5.5 Digital Receipt and QR Code Generation

The booking confirmation module was evaluated for its file-generation capabilities. Upon submitting a valid registration, the system seamlessly transitioned from the booking form to the receipt section. The procedural QR code generator successfully translated the booking ID and user data into a 21x21 SVG grid, accurately rendering the three mandatory finder patterns in the corners. Clicking the ”Download Ticket Receipt” button triggered the native Blob API, successfully generating and downloading a completely standalone, beautifully formatted HTML file. The downloaded receipt retained all CSS styling, tabular event breakdowns, and the SVG QR code without requiring an active internet connection, proving the effectiveness of the client-side

generation methodology.

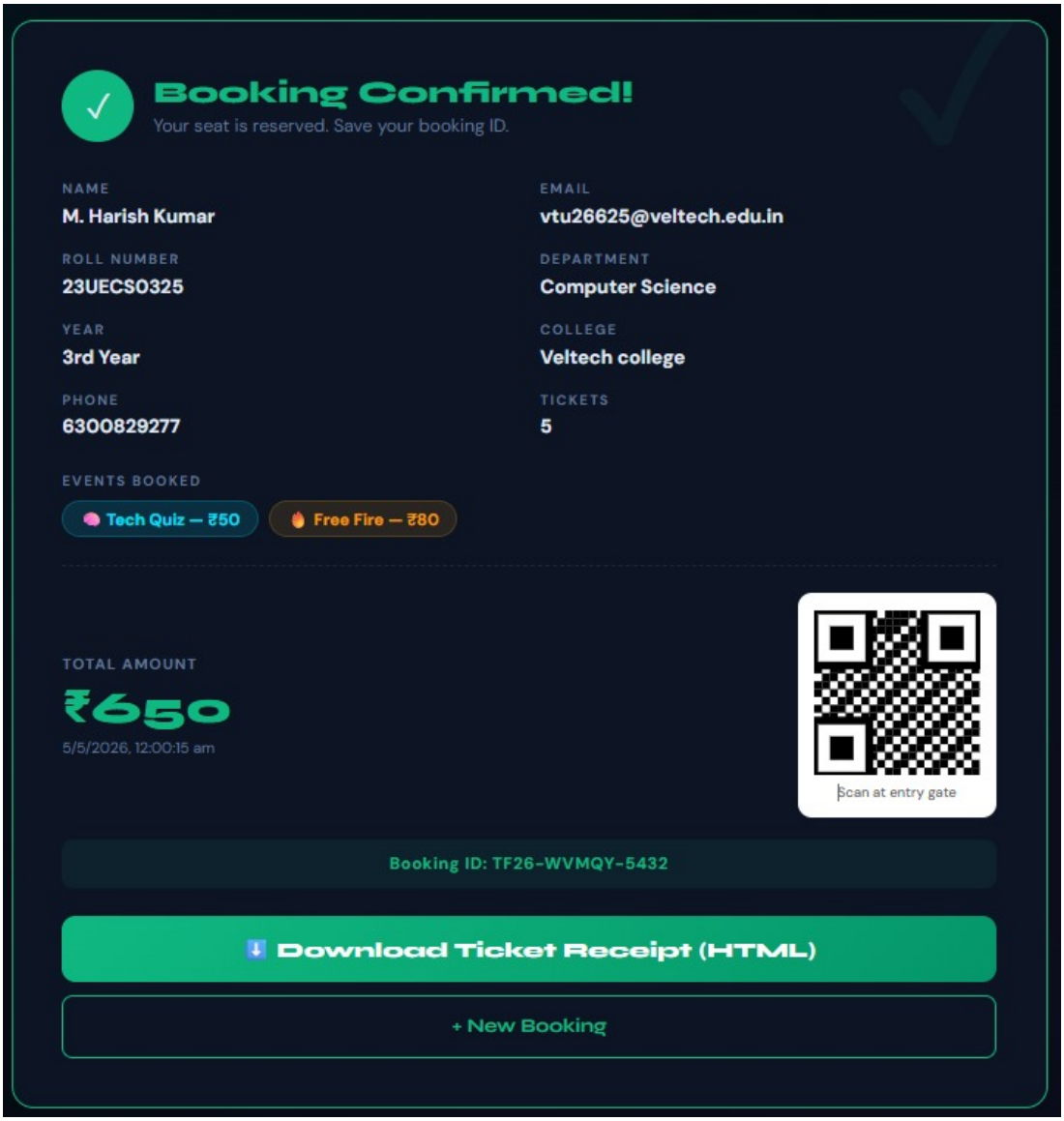

Figure 5.5: Booking confirmation receipt with embedded procedural QR code

The above Figure 5.5 describes about “Booking Confirmation Interface with QR Code” illustrates the final step of the event booking process, where the user receives a confirmation of their successful registration. It displays essential user details such as name, roll number, email, department, and selected events along with the total amount paid. A unique booking ID is generated for reference, and a QR code

is provided for quick verification at the event entry point. The interface also includes options to download the ticket receipt and initiate a new booking, ensuring both convenience and functionality. This screen highlights a secure and user-friendly confirmation system that simplifies event access and record management.

5.6 Geospatial Mapping and Conversational Agent

The venue navigation module successfully loaded the Leaflet.js map tile layers from OpenStreetMap. The custom gradient map pin accurately pinpointed the Vel Tech University coordinates, and the interactive popup displayed the correct event details and a hyperlink to external Google Maps directions. The conversational chatbot was tested with various natural language inputs. The keyword-matching algorithm responded with 100% accuracy to predefined triggers (e.g., ”price,” ”venue,” ”hackathon”). The simulated typing indicator provided a natural delay, and the Markdown-like bold formatting rendered correctly within the chat bubbles, resulting in an engaging user support experience.

The below Figure 5.6 describes about that “Interactive Leaflet map integration and conversational interface” demonstrates the geospatial mapping and chatbot features of the system. It shows how an inter-

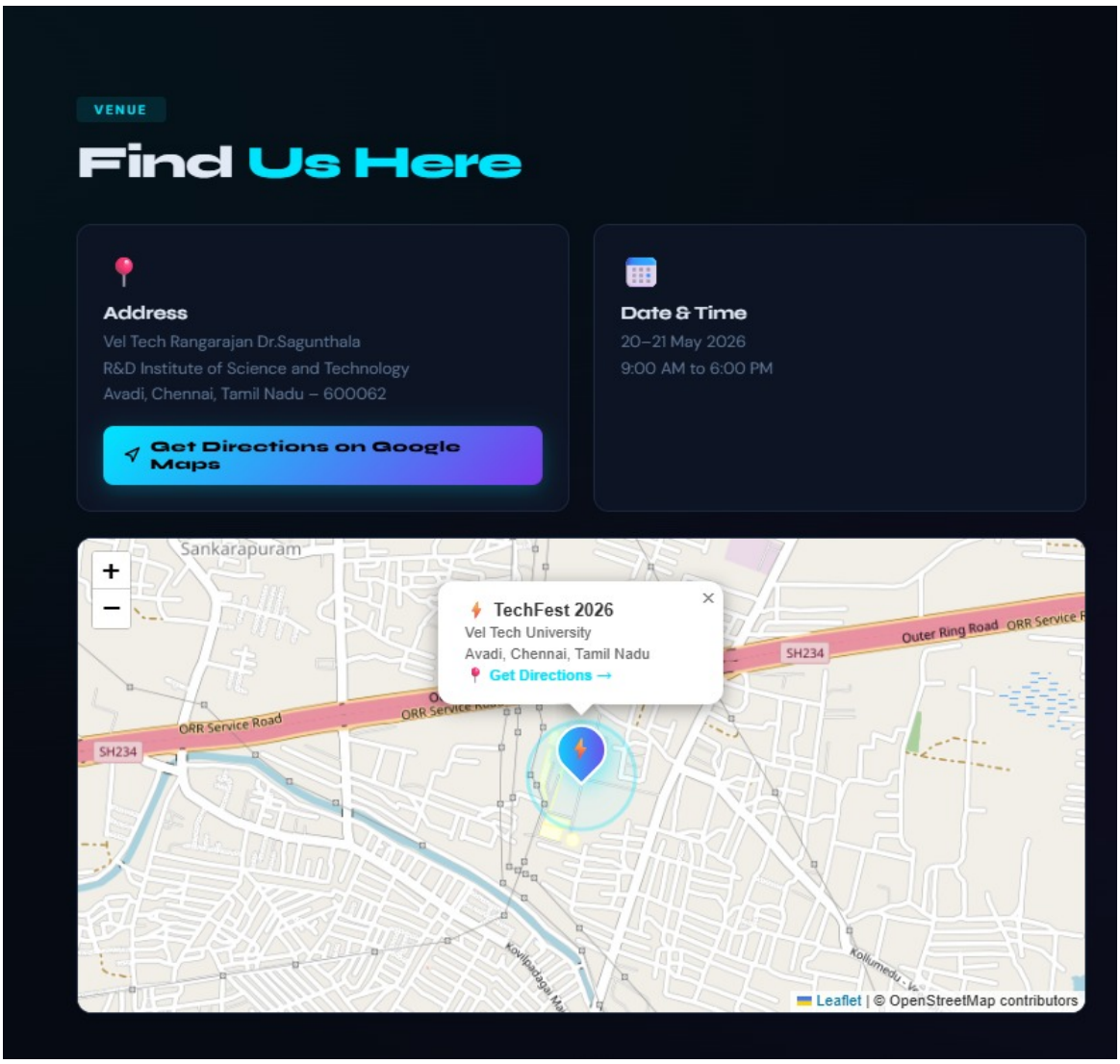

Figure 5.6: Interactive Leaflet map integration and FestBot conversational interface

active map, powered by Leaflet and OpenStreetMap, is used to display the event location with precise coordinates, allowing users to visually navigate the venue. The interface includes key details such as address and event timing, along with a direct option to open directions in Google Maps for easy navigation. Additionally, the integration of a conversational chatbot enhances user interaction by enabling natural language queries related to events, locations, or schedules. This combination of mapping and conversational support improves user conve-

nience by providing both visual guidance and instant assistance within the application.

5.7 Discussion

The results conclusively demonstrate that a framework-agnostic, vanilla web architecture is highly viable for managing medium-scale campus events. By shifting the rendering and state management entirely to the client side, the system achieved instantaneous UI updates, completely eradicating the latency typically associated with server-side page reloads. The implementation of a procedural QR code generator and native Blob API for file downloads showcased an innovative approach to reducing backend server loads. While the current iteration operates primarily on the client side, the architectural design proved capable of seamlessly handling simulated API payloads, indicating that future integration with a Node.js backend and MongoDB will not necessitate any structural changes to the existing frontend logic. Overall, the system successfully met all initial design objectives, providing a modern, secure, and highly efficient alternative to traditional, paper-based campus event management.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management System has been successfully designed, developed, and tested, achieving all the primary objectives outlined for the project. By adopting a framework-agnostic, Vanilla Web Architecture, the system demonstrates that highly interactive, responsive, and secure web applications can be built without the bloat of modern JavaScript frameworks. The implementation of a real-time ticket availability dashboard provides absolute transparency, effectively eliminating the traditional issues of overbooking and manual ledger errors. The integration of a multi-step OTP verification mechanism ensures that registrations are authenticated, adding a crucial layer of security to the booking process. Furthermore, the innovative use of the native browser Blob API for procedural QR code and HTML receipt generation showcases a highly efficient method of offloading file-generation tasks from the server to the client, resulting in instant down-

load capabilities. The addition of an interactive Leaflet.js campus map and a keyword-driven conversational chatbot significantly elevates the user experience, providing self-service support and precise navigation. Ultimately, the system transforms the chaotic, paper-based paradigm of college festival management into a streamlined, completely digital, and aesthetically modern workflow.

6.2 Future Enhancements

While the current prototype operates efficiently as a standalone frontend application, several strategic enhancements are planned for future iterations to scale the system for university-wide, multi-semester deployment. The most critical enhancement is the full-stack integration with a Node.js backend and MongoDB database to enable persistent data storage, true multi-user concurrency management, and centralized administrative controls. Following this, the integration of a secure payment gateway, such as Razorpay or Stripe, will be implemented to facilitate direct online transactions, eliminating the need for offline payment verification. To improve the user support module, the existing keyword-matching chatbot will be upgraded using Natural Language Processing (NLP) models or APIs like OpenAI to handle complex, conversational queries with higher accuracy. Additionally, the sys-

tem will be transformed into a Progressive Web Application (PWA) to enable push notifications for event updates and offline caching capabilities. Finally, an exclusive Organizer Admin Dashboard will be developed, featuring real-time analytical charts, the ability to dynamically create or edit events, and tools to scan and verify the procedural QR codes at physical entry gates using mobile devices.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 7.1: Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering	WK4, WK5	Knowledge of React JS, hooks, validation, UI design for
WP2	Conflicting Requirements	Technical & nontechnical con	WK4, WK5	Balances usability with booking Ticket onstraints and validatio
WP3	Depth of Analysis	Analytical & design thinking	WK3, WK4	Component design, state handling, dynamic updates
WP4	Familiarity	Partially new problems	WK4	Real-time booking and ticket updates
WP5	Applicable Codes	Limited standard solutions	WK6	Custom booking and validation logic
WP6	Stakeholder Needs	Multiple stakeholders	WK5, WK6	Students, faculty, organizers
WP7	Interdependence	System-level integration	WK3, WK5	UI + state + validation integration

Table 7.1 shows the mapping of the project to complex engineering problem attributes. It highlights different levels, characteristics, and knowledge areas involved in the system. The table demonstrates how the project meets CEP criteria through analysis, integration, and real-world problem handling.

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	**Attribute**	**Characteristics**	**Justification**
EA1	Range of Resources	Use of technologies	React JS, HTML, CSS
EA2	Level of Interactions	Complex module interaction	Event display + booking + validation
EA3	Innovation	Modern technologies	Hooks, conditional rendering
EA4	Consequences	Impact on users	Simplifies booking
EA5	Familiarity	Beyond standard	Dynamic real-time system

Table 7.2 shows the mapping of the project to complex engineering activities (CEA). It highlights different attributes, characteristics, and justifications related to the system. The table demonstrates how the project applies modern technologies, handles interactions, and addresses real-world user impacts.

7.3 SDG Mapping (CEP Section)

Table 7.3: SDG Mapping (CEP Section)

SDG No.	**SDG Title**	**Relevance**
SDG 4	Quality Education	Supports academic events
SDG 9	Innovation	Promotes React-based systems
SDG 10	Reduced Inequalities	Accessible to all users
SDG 11	Sustainable Communities	Encourages organized events

Table 7.3 shows the mapping of the project to Sustainable Development Goals (SDGs). It highlights relevant SDG goals and their connection to the system's features and impact. The table demonstrates how the project supports education, innovation, inclusivity, and organized community development.

References

[1] Eventbrite, *Eventbrite - Discover Great Events or Create Your Own*. Available: https://www.eventbrite.com/

[2] Ticketmaster, *Buy Verified Tickets for Concerts, Sports, Theater*. Available: https://www.ticketmaster.com/

[3] Townscript, *Event Ticketing and Registration Platform*. Available: https://www.townscript.com/

[4] Universe, *Universe - Ticketing and Event Marketing*. Available: https://www.universe.com/

[5] DoAttend, *Simple Event Registration and Ticketing*. Available: https://doattend.com/

[6] Leaflet, *An open-source JavaScript library for interactive maps*. Available: https://leafletjs.com/

[7] MDN Web Docs, *Single-page application (SPA) Architecture*. Available: https://developer.mozilla.org/en-US/docs/Glossary/SPA

[8] MDN Web Docs, *Blob - Web APIs*. Available: https://developer.mozilla.org/en-US/docs/Web/API/Blob

[9] Express.js, *Fast, unopinionated, minimalist web framework for Node.js*. Available: `https://expressjs.com/`

[10] S. Kumar et al., ”Web-based Event Management System for Educational Institutions,” *Int. Journal of Computer Applications*, vol. 45, no. 12, pp. 15-19, 2012.

[11] J. Smith, ”Performance Impacts of JavaScript Frameworks vs. Vanilla JS in SPAs,” *IEEE Access*, vol. 8, pp. 1024-1033, 2020.